

Hackathon Video Presentation Script:

SysPilot CausalTrace

Presentation Style: Confident, Visionary, TED-Talk / Apple Keynote

Target Duration: ~5 Minutes

Audience: Judges, Engineers, Non-Specialists

1. Opening Hook (15–30 Seconds)

		NARRATION SCRIPT	
VISUALS	ANIMATIONS & TRANSITIONS	KEY MESSAGE	
A black screen. A large, neon-red timer appears in the center, counting down rapidly: 03:14... 03:13... 03:12... Beneath it, the words "PRODUCTION DATABASE DOWN" flash in white.	Fade in from black. The timer numbers pulse with a slow, heartbeat-like glow.	"It is 3:00 AM. Your phone is buzzing off the hook. A critical database in your application has completely frozen. Customers are getting checkout errors. Your startup is losing money by the second. You open your system dashboard, and everything looks... normal. The database shows 0% CPU, no disk writes, and no errors. It has silently vanished into a black box. This is the ultimate developer nightmare."	Modern software systems can fail silently, and traditional tools are blind to the cause.

2. Problem Story (30–60 Seconds)

VISUALS	ANIMATIONS & TRANSITIONS	NARRATION SCRIPT	KEY MESSAGE
Split screen. On the left: a developer drowning in a wall	Smooth slide transition from the	"When systems freeze like this, we reach for our trusty tools: debuggers, log files, or performance charts. But traditional	Traditional monitoring tools give us raw

of raw terminal text

logs. On the right: a

simple graphic of a hook. The text on

house with a locked the left scrolls at an

front door. Inside, unreadable speed.

water is leaking, but The house on the

the plumber is right slowly fills with

locked outside cartoon water.

because the keys

are lost.

| *dashboards only show us aggregates—like*

| *total memory or average CPU. They don't*

| *show relationships. It's like checking the*

| *weather inside a house when a pipe is*

| *leaking behind the drywall. In Linux, when a*

| *parent program spawns a child, it is*

| *supposed to clean up after itself. But often, it*

| *forgets to set a tiny safety flag called 'close-*

| *on-exec'. The child silently inherits a massive*

| *keyring of locked vaults it shouldn't even*

| *know exist. When the parent starts twisting*

| *keys, the child's locks jam shut. The database*

| *freezes, but to the outside, it looks*

| *completely idle. You are left sorting through*

| *gigabytes of raw logs, trying to find one*

| *missing key in a dark room."*

numbers, but they

don't show the

hidden

relationships that

cause system

lockups.



3. Introducing the Project: SysPilot CausalTrace (30 Seconds)

VISUALS

The split screen slides away.

A sleek, dark-blue title card

fades in: **SysPilot**

CausalTrace. A graphic of a

glowing, interactive map of

process relationships

appears, with nodes

representing programs and

lines representing

connections.

ANIMATIONS &

TRANSITIONS

Cinematic zoom-in

on the title. The

nodes on the map

light up one by one

in blue, yellow, and

red.

NARRATION SCRIPT

| *"We built SysPilot CausalTrace to*

| *solve this. SysPilot is a lightweight,*

| *local operating system reasoning*

| *agent. Think of it as a smart GPS for*

| *your system's hidden plumbing.*

| *Instead of showing you a wall of*

| *text, SysPilot dynamically constructs*

| *a live map of your system's*

| *processes and resources. It tracks*

| *who spawned whom, who is writing*

| *to what, and exactly where the keys*

| *are leaking—so you can find the*

| *blockage instantly."*

KEY MESSAGE

SysPilot

CausalTrace is an

intelligent,

relationship-aware

map for

diagnosing system

blockages.



4. How It Works (1–2 Minutes)

NARRATION SCRIPT			
VISUALS A three-step storyboard animation: 1. Kernel Camera: A small camera icon sits above a Linux kernel ring, taking snapshots. 2. The Web: The snapshots connect to form a web of circles (processes) and lines (links). 3. The Detective: A magnifying glass follows a line backward from a red circle to a yellow circle.	ANIMATIONS & TRANSITIONS Fade transitions between the three steps. The lines in step 2 pulse to show data flowing.	"How does SysPilot do this without slowing down your computer? It uses three core pillars: First: Kernel Cameras (eBPF). Instead of tracing programs from the outside, we use a Linux technology called eBPF. It's like placing high-speed cameras inside the kernel's hallways. Every time a program opens a file or starts a new task, the camera snaps a photo with near-zero overhead. Second: The Connected Web. SysPilot takes these snapshots and builds a directed graph. In simple terms, it draws a map. Processes are cities, and files or sockets are the roads connecting them. We mark the relationships: who is writing, who is reading, and who is blocked. Third: The Detective (Reverse-BFS). When your database freezes, SysPilot acts as a detective. It starts at the frozen node and walks backward along the roads. It bypasses the hundreds of idle roads and traces the route directly to the rogue parent process that leaked the key."	
		KEY MESSAGE By combining lightweight kernel cameras with a smart detective algorithm, SysPilot traces problems to their source without slowing down the machine.	

⚡ 5. What Makes It Different (1 Minute)

A simple, clean comparison screen.		<i>"So, why can't you just use existing tools? APM dashboards and cloud monitors show you that a server is slow, but they can't tell you the line of code that caused it. Profilers tell you what functions are running, but they can't see relationship leaks across different programs. And running deep tracers in production is like putting speed bumps on a highway—it slows your system down tenfold. SysPilot is different. It runs entirely locally on your machine, protecting your database credentials and privacy. It has near-zero overhead. And best of all, it connects the telemetry map directly to a local AI brain. It doesn't just show you a chart; it tells you exactly what line of code to fix."</i>	
- Left side: "Old Way" (A massive pile of paper logs, a slow magnifying glass, and label: "10x Latency Slowdown").	TRANSITIONS A sliding comparison line moves from left to right, revealing the clean SysPilot interface.		SysPilot is local, introduces no performance penalties, and tells you how to fix the code, not just that a problem exists.
- Right side: "SysPilot Way" (A glowing 3D map, a local AI assistant, and badge: "Zero Overhead, 100% Private").			

6. Live Demo Narrative (1–2 Minutes)

VISUALS	ANIMATIONS & TRANSITIONS	NARRATION SCRIPT	KEY MESSAGE
Screen recording of the terminal. We see the user type <code>syspilot explain --pid 40126 --causal --no-index</code> and press Enter. The terminal streams text. Then, the view switches to a beautiful, dark-mode browser dashboard. Circles bounce and float dynamically, and a red path highlights the connection between <code>mock_db</code> and <code>language_server</code> .	TRANSITIONS Fade to screen recording. Zoom in on the browser dashboard as the cursor hovers over the red node, opening a sidebar with code fixes.	<i>"Let's see it in action. Here, our database, <code>mock_db</code>, is frozen. We run <code>syspilot</code> explain on its process ID. Immediately, the local AI agent analyzes the system and outputs the causal chain. It tells us that <code>mock_db</code> is trapped because its parent, the language server, leaked a file lock when spawning a background task. Now, we switch to our interactive web dashboard. Here is the D3 force-directed view of our system. Notice the red path: it highlights the bottleneck running from our frozen database directly back to the language server. We click on the culprit node, and the sidebar displays the code remediation. SysPilot doesn't just tell us to kill the process; it shows us the exact</i>	The interactive visual dashboard turns complex system relationships into a simple, clickable map with instant code fixes.

| JavaScript code change needed in the
| language server's spawn settings to
| ensure child processes ignore standard
| descriptors. We apply the fix, restart, and
| the system is running smoothly."

7. Impact & Future Vision (30–60 Seconds)

VISUALS

A clean slide showing a network of multiple servers connecting together. Icons for "Distributed Networks", "Autonomous Systems", and "Self-Healing Infrastructure" light up.

ANIMATIONS & TRANSITIONS

Nodes expand outwards, creating a global web representation.

NARRATION SCRIPT

| "SysPilot matters because software
| architecture is getting more distributed
| every day. In a world of microservices
| and containers, tracking resource leaks
| manually is a losing battle. Beyond this
| hackathon, we want to expand SysPilot
| to support distributed network
| environments. Imagine an autonomous
| system that doesn't just diagnose leaks,
| but automatically reorganizes resource
| priorities and heals itself in real-time
| when a bottleneck is detected. We are
| building the foundation for self-healing
| systems."

KEY MESSAGE

SysPilot is laying the groundwork for self-healing, autonomous infrastructure.

8. Closing Statement (15 Seconds)

VISUALS

The screen fades to black. In the center, a simple logo of SysPilot shines in white, next to the tagline: "Observability with Causality."

ANIMATIONS & TRANSITIONS

Logo glows gently and slowly fades out.

NARRATION SCRIPT

| "Debugging shouldn't be about
| grepping through logs at 3:00
| AM. Stop guessing, and start
| seeing the connections. Let
| CausalTrace pilot your systems
| diagnostics. Thank you."

KEY MESSAGE

CausalTrace is the future of automated, relationship-aware system diagnostics.